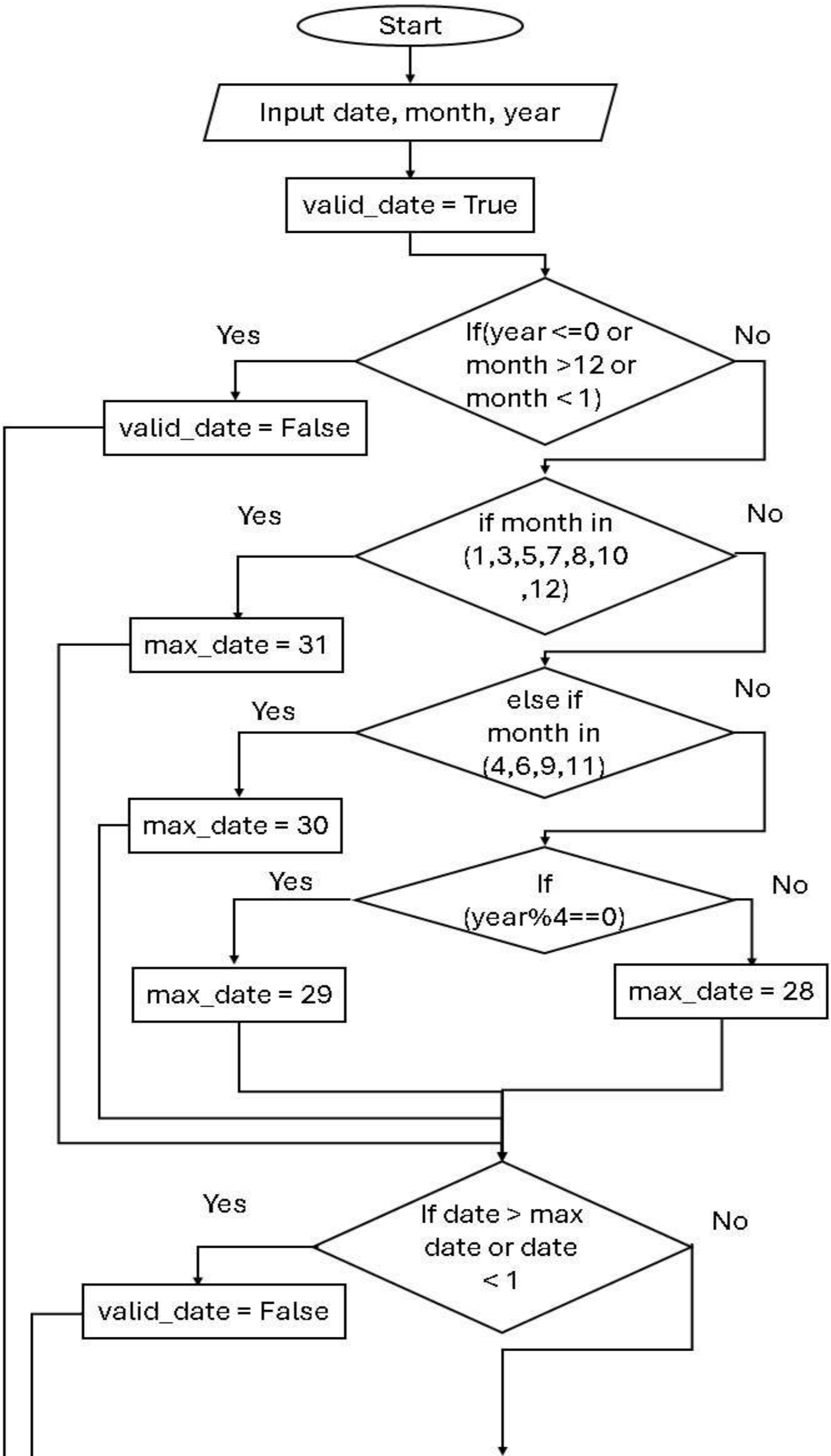
Experiment 6

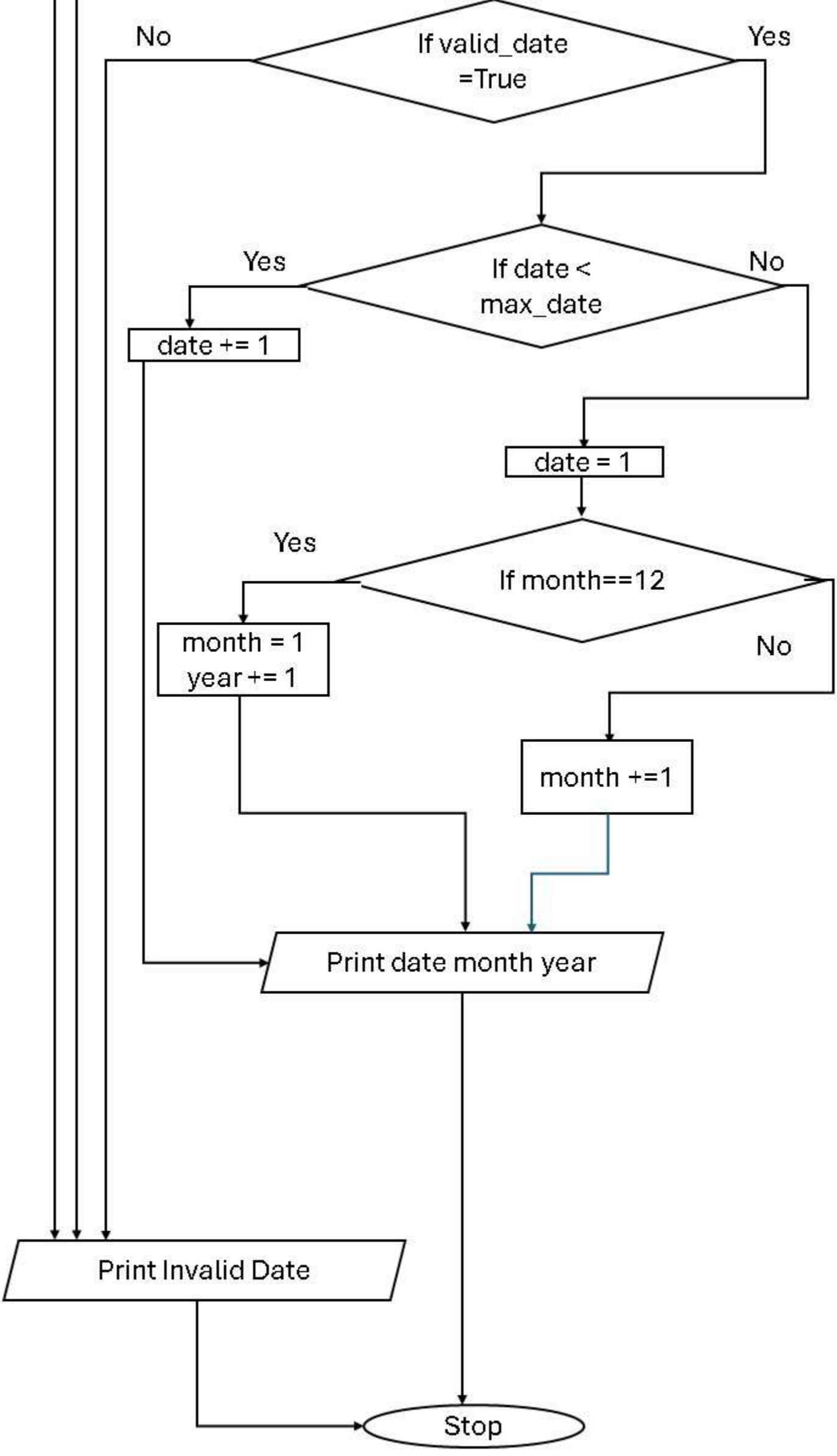
6.1.1 Incremented Date

Algorithm :

```
Step 1: Start
Step 2: Input date, month, year
Step 3: valid_date = True.
Step 4: If (year <= 0 or month >
12 or month < 1)
                then valid_date =
False
Step 5: If month is
1,3,5,7,8,10,12
                then max_date = 31
                 else if month is
4,6,9,11
                then max_date = 30
                 else
                   if year % 4 ==
0
                         max_date
= 29
                   else
                         max_date
= 28
Step 6: If (date > max_date or
date < 1)
                then valid_date =
False
Step 7: If valid_date = True
                   if date <
max_date
                          date =
date + 1
                else
                         date = 1
```

Flowchart :


Start
Input date, month, year
valid_date = True
If(year <=0 or month >12 or month < 1)
Yes
No
valid_date = False
if month in (1,3,5,7,8,10 ,12)
Yes
No
max_date = 31
else if month in (4,6,9,11)
Yes
No
max_date = 30
If (year%4==0)
Yes
No
max_date = 29
max_date = 28
If date > max date or date < 1
Yes
No
valid_date = False


If valid_date =True
No
Yes
If date < max_date
Yes
No
date += 1
date = 1
If month==12
Yes
No
month = 1
year += 1
month +=1
Print date month year
Print Invalid Date
Stop

Code :

```
date = int(input())
month = int(input())
year = int(input())

flag = True

if month < 1 or month > 12 or year <
0:
    flag = False

if flag:
    if month in (1, 3, 5, 7, 8, 10,
12):
        max = 31
    elif month in (4, 6, 9, 11):
        max = 30
    elif month == 2:
        if year % 4 == 0:
            max = 29
        else:
            max = 28
    else:
        flag = False

if flag:
    if month == 12 and date == 31:
        date = 1
        month = 1
        year += 1
    elif date == max:
        month += 1
        date = 1
    elif date < max:
        date += 1
    else:
        flag = False

if flag == False:
    print("Invalid Date")
else:
    print(f"{date:02d}-{month:02d}-
{year}")
```

6.1.1. Incremented Date

45:26

Write a Python program to check if a given date is valid. If the date is valid, print the next day's date (incremented date). If the date is invalid, print "Invalid Date".

Date Validation Rules:

1. **Valid Month**: 1 to 12
2. **Valid Day**: Depends on the month and year
   - January (1), March (3), May (5), July (7), August (8), October (10), December (12): 1 to 31 days
   - April (4), June (6), September (9), November (11): 1 to 30 days
   - February (2): 1 to 29 days in a leap year and 1 to 28 days in a non-leap year
3. **Valid Year**: Any positive integer greater than zero.

Date Increment Rules:

If the date is valid, increment it by one day:

- If it's the last day of the month, move to the 1st day of the next month
- If it's December 31st, move to January 1st of the next year

Input Format:

- First line contains an integer representing the day
- Second line contains an integer representing the month
- Third line contains an integer representing the year

Output Format:

- If the date is valid, print the incremented date in the format:

```
<DD>-<MM>-<YYYY>
```

- If the date is invalid, print: "Invalid Date".

Sample Test Cases +

Explorer

nextDate.py

Submit

```python
date = int(input())
month = int(input())
year = int(input())

flag = True

if month <= 0 or month > 12 or year < 0:
    flag = False

if flag:
    if month in (1, 3, 5, 7, 8, 10, 12):
        max_days = 31
    elif month in (4, 6, 9, 11):
        max_days = 30
    elif month == 2:
        if year % 4 == 0:
            max_days = 29
        else:
            max_days = 28
    else:
```

Debugger

Average time	Maximum time
0.010 s	0.018 s
10.30 ms	18.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 18 ms Debug

Expected output	Actual output
15	15
3	3
2024	2024
16-03-2024	16-03-2024

Test case 2 13 ms

Terminal Test cases